IMSET
INSTITUT MAGHRÉBIN DES SCIENCES ÉCONOMIQUES ET DE TECHNOLOGIE
HONORIS UNITED UNIVERSITIES

ACADÉMIE D'ART DE CARTHAGE
HONORIS UNITED UNIVERSITIES

Intitulé du cours: Algorithmique et Programmation

Chapitre 1 : Introduction à l'algorithmique

Filière: 1ère année – BTP-Soutien en Informatique de Gestion

Enseignante: Abid Arij

tt_arij.abid@imset.tn

Année de formation 2024/2025

Plan

- Objectifs du module
- Qu'est-ce qu'un algorithme ?
- Pourquoi apprendre l’algorithmique ?
- Structure générale d’un algorithme
- Algorithmique : Notions de base
- Les Variables
- Les Constantes
- Les instructions d’entrées / Sorties

Objectifs du module

À la fin du module, l'apprenant doit être capable de :

- Comprendre ce qu’est un algorithme
- Apprendre à concevoir et représenter des algorithmes
- Résoudre des problèmes simples par la méthode algorithmique
- Apprendre les concepts de base de l’algorithmique et de la programmation.
- Être capable de mettre en œuvre ces concepts pour analyser des problèmes simples et écrire les algorithmes correspondants.

algorithme

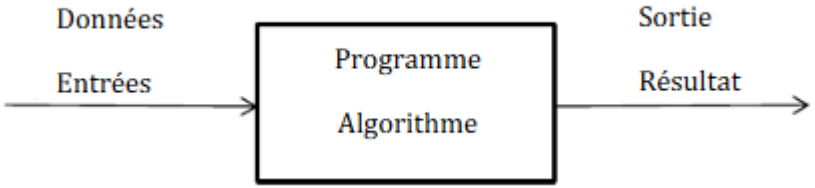
01

Qu'est-ce qu' un algorithme ?

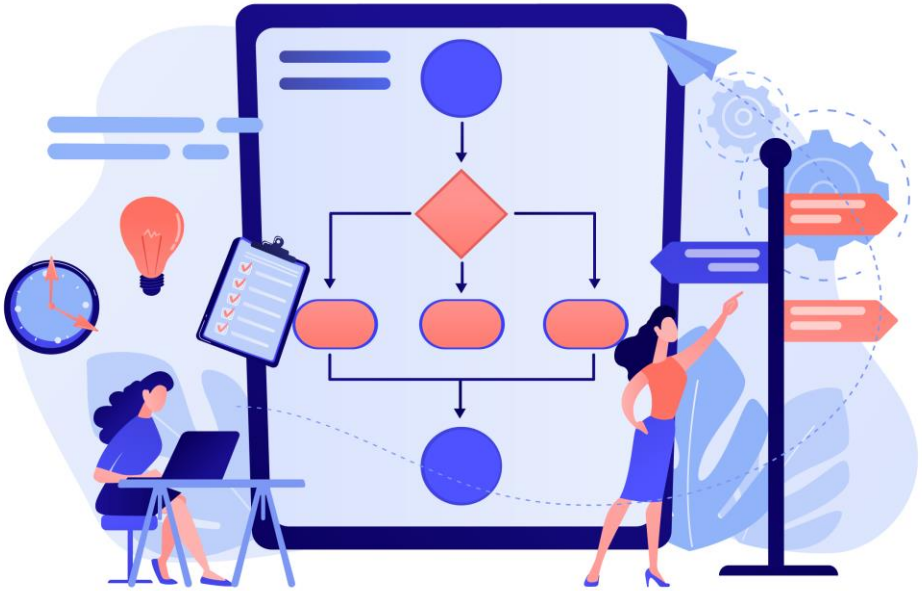
Un **algorithme** est une suite finie et ordonnée d'instructions qui permet de résoudre un problème ou de réaliser une tâche.
C'est une recette logique, comme un mode d'emploi que l'on peut suivre étape par étape pour obtenir un résultat précis.

Chaque étape de l'algorithme doit être :

- **Clair** : compréhensible pour celui qui l'utilise
- **Précis** : sans ambiguïté
- **Fini** : il doit y avoir un nombre limité d'étapes
- **Exécutable** : réalisable concrètement



- **Algorithme** = méthode de résolution.
- L’**algorithmique** désigne aussi la discipline qui étudie les algorithmes et leurs applications en **informatique**.
  Une bonne connaissance de l’algorithmique permet d’écrire des algorithmes exacts et efficaces.



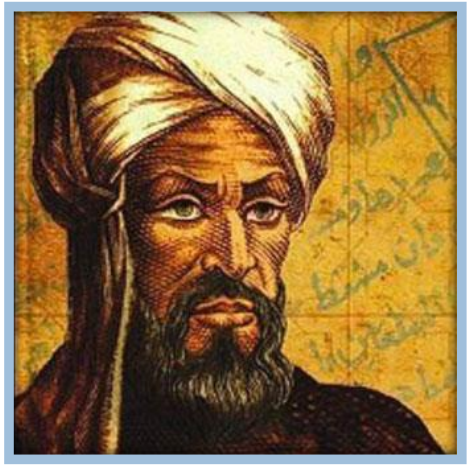
D'où vient le mot "Algorithme" ? Origines Historiques

Le mot **algorithme** vient du nom d'un grand savant :
Al-Khwarizmi (vers 780 - 850), un mathématicien persan qui a vécu à Bagdad.

Il a écrit un livre intitulé *"Le calcul indien"* dans lequel il expliquait des méthodes de calcul avec les chiffres que nous utilisons aujourd'hui (appelés chiffres arabes).

Son nom a été latinisé en **Algoritmi**, et c'est ainsi que le mot "algorithme" est né.

Al-Khwarizmi est donc considéré comme le **père de l'algorithmique**.



Pourquoi apprendre l'algorithmique ?

Apprendre l'algorithmique est essentiel pour plusieurs raisons :

1. Développer une pensée logique : L'algorithmique aide à structurer votre pensée et à résoudre des problèmes de manière méthodique.

2. Programmer des ordinateurs : Tous les programmes informatiques sont basés sur des algorithmes. Apprendre à concevoir des algorithmes est donc une étape clé pour devenir développeur.

3. Comprendre le monde numérique : Dans un monde où la technologie joue un rôle central, comprendre les algorithmes permet de mieux saisir comment fonctionnent les outils numériques.

4. Résoudre des problèmes complexes : Que ce soit en science, en ingénierie ou en gestion,

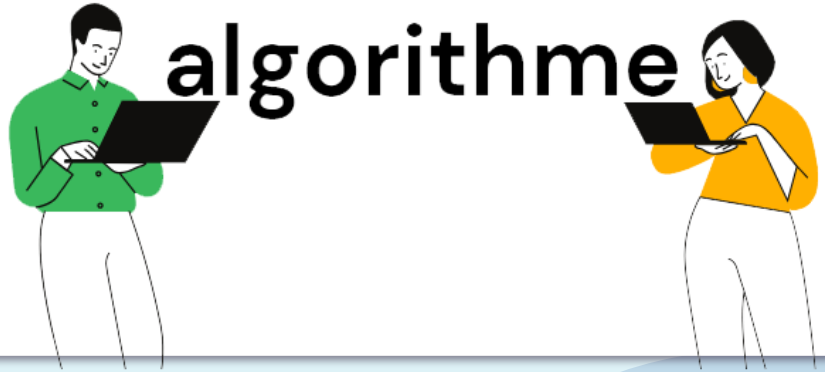
   les algorithmes sont utilisés pour résoudre des problèmes difficiles.

Exemples d'utilisation :

- Calculer une moyenne d'étudiants
- Recherche d'un mot dans un texte
- Tri d'une liste de noms
- Systèmes GPS (trouver le chemin le plus court)
- Trier des données dans un tableau

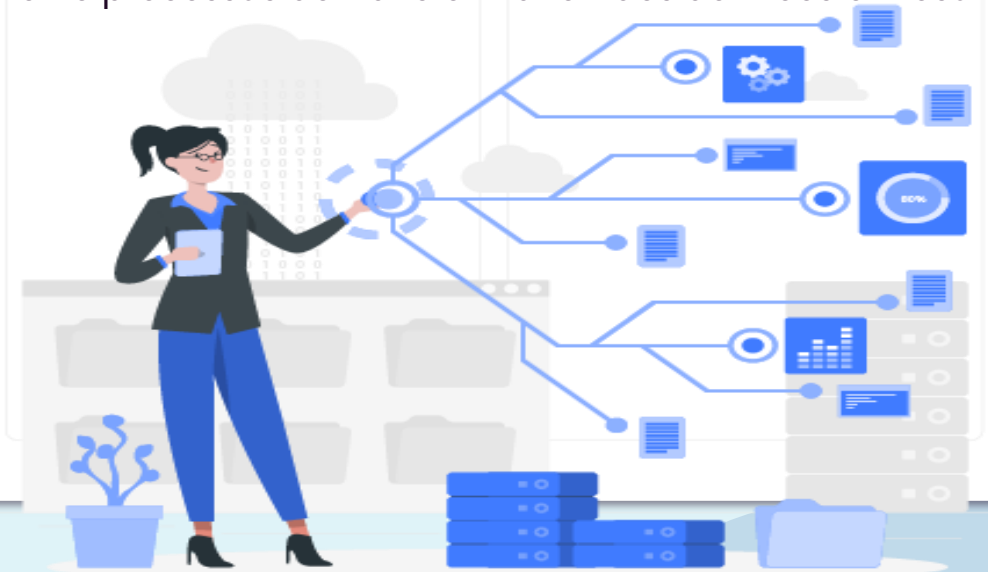
Un algorithme doit :

- avoir un nombre fini d'étapes, avoir un nombre fini d'opérations par étape, se terminer après un nombre fini d'opérations, fournir un résultat.
- Les séquences (étapes) se succèdent dans un certain ordre.
- Chaque opération doit être définie rigoureusement et sans ambiguïté.
- Un algorithme est caractérisé par un début et une fin.
- Le comportement d'un algorithme est déterministe.



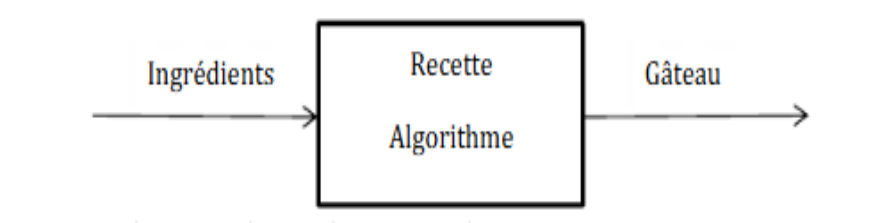
La résolution d'un problème se caractérise par quatre étapes :

1. Comprendre la nature du problème posé.
2. Préciser les données fournies (Entrées).
3. Préciser les résultats que l'on désire obtenir (Sorties).
4. Déterminer le processus de transformation des données en résultats.



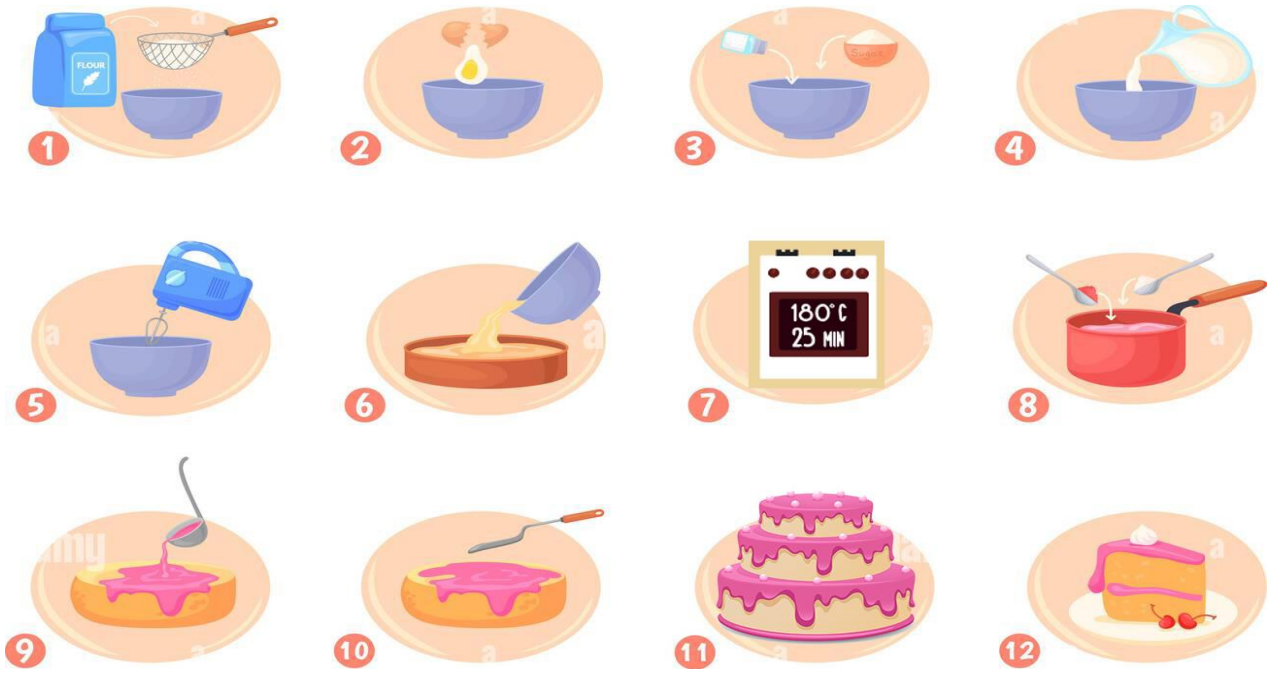
Un algorithme peut être comparé à une recette de cuisine :

- **Le résultat,** c'est comme le plat à cuisiner.
- **Les données** sont l'équivalent des ingrédients de la recette.
- **Les règles** de transformation correspondent aux directives ou instructions de la recette.



Ingrédients du Gâteau
FLOUR
milk

Ces étapes forment **un algorithme** pour préparer **un gâteau**.
Si vous suivez ces instructions dans l'ordre, vous obtiendrez **un gâteau**.
Si vous changez **l'ordre des étapes, le résultat** ne sera pas celui attendu.



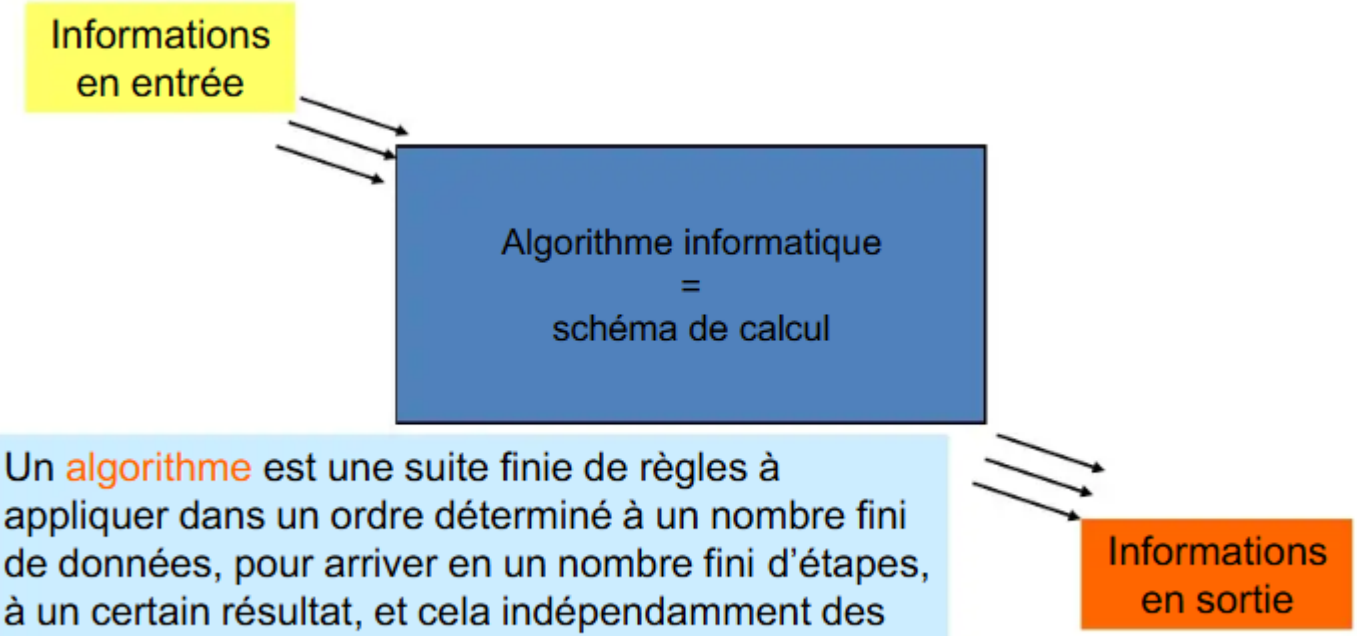
Algorithme : Préparer un gâteau

Algorithme : Préparer un gâteau
Début

Farine
Sucre
Œufs
Beurre
Levure
Lait
Dans un saladier, mélanger la farine, le sucre et la levure.

Ajouter les œufs un par un en mélangeant.
Faire fondre le beurre et l'ajouter au mélange.
Verser le lait et bien mélanger jusqu'à obtenir une pâte homogène.
Préchauffer le four à 180°C.
Verser la pâte dans le moule.
Enfourner pendant 30 à 40 minutes
Vérifier la cuisson avec la lame d'un couteau
Sortir le gâteau du four et **laisser refroidir**.
Fin

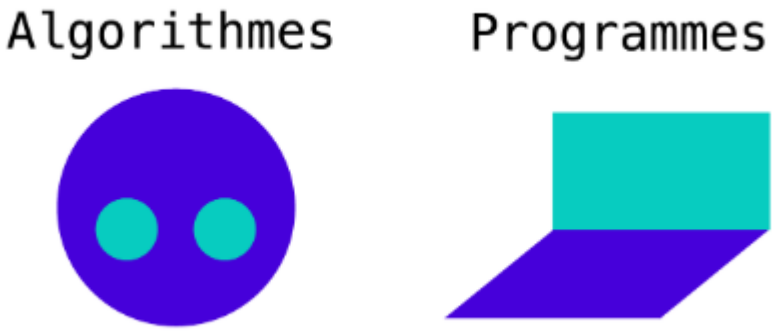
Algorithmes et programmes



Un algorithme est une suite finie de règles à appliquer dans un ordre déterminé à un nombre fini de données, pour arriver en un nombre fini d'étapes, à un certain résultat, et cela indépendamment des données.

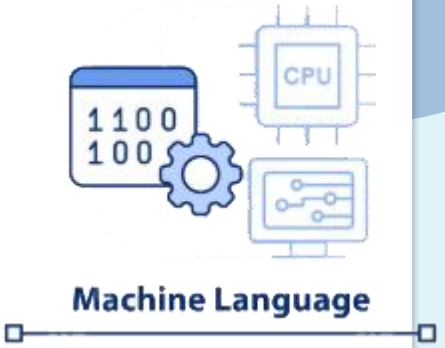
Quelle est la différence entre un algorithme et un programme ?

- Un **algorithme** est une idée, une méthode logique pour résoudre un problème. Il est écrit en langage humain ou en pseudo-code.
- Un **programme** est la traduction de cet algorithme dans un langage de programmation que l'ordinateur peut comprendre (Python, C, Java, etc.).



- Programme :
  - codage d'un algorithme afin que l'ordinateur puisse exécuter les actions décrites
  - doit être écrit dans un langage compréhensible par l'ordinateur
    - → langage de programmation
- Un programme est donc une suite ordonnée d'instructions élémentaires codifiées dans un langage de programmation

- RAPPEL: Langage machine
  - langage binaire
  - ses opérations sont directement compréhensibles par l'ordinateur
  - propre à chaque famille d'ordinateur



- Ecriture des premiers programme en langage machine

Algorithme

↓

programmation

↓

Programme en langage évolué

↓

traduction

↓

Programme en langage machine

↓

Interprétation par l'Unité Centrale de traitement

↓

le traitement souhaité est réalisé

Organigramme / Pseudo-code :

Historiquement, il y a deux façons pour représenter un algorithme :

- **L'Organigramme :** représentation graphique avec des symboles (carrés, losanges, etc.). Offre une vue d'ensemble de l'algorithme. Représentation quasiment abandonnée aujourd'hui.

Le Pseudo-code : représentation textuelle avec une série de conventions ressemblant à un langage de programmation (sans les problèmes de syntaxe).
Plus pratique pour écrire un algorithme. Représentation largement utilisée.

Exemple 1 : Afficher un message

🗣️ **Langage naturel :**

Afficher le message “Bonjour !”

💻 **Pseudo-code :**

```
Début
   Afficher "Bonjour !"
Fin
```

Exemple 2 : Lire un nombre et l'afficher

🗣️ **Langage naturel :**

Demander à l'utilisateur un nombre, puis l'afficher.

💻 **Pseudo-code :**

```
Début
   Lire (nombre)
   Afficher nombre
Fin
```

À quoi sert un algorithme ?

-Résoudre des problèmes

Les algorithmes sont utilisés pour résoudre des problèmes **dans divers domaines :**

- **Mathématiques :** Calculer des racines carrées, résoudre des équations.
- **Informatique :** Trier des données, chercher des informations dans une base de données.
- **Dans les sciences :** Les algorithmes sont utilisés pour modéliser des phénomènes naturels, analyser des données expérimentales, ou simuler des réactions chimiques.
- **Vie quotidienne :** Planifier un itinéraire (Google Maps), recommander des films (Netflix)

pour organiser son emploi du temps, choisir le chemin le plus court pour aller au travail,

ou encore trier ses vêtements par couleur.

-Automatiser des tâches

Les algorithmes permettent aux ordinateurs d'effectuer des tâches répétitives ou complexes sans intervention humaine. Par exemple :

- Un algorithme peut automatiser le traitement des emails pour détecter les spams.
- Un algorithme peut analyser des millions de transactions bancaires pour détecter des fraudes.

-Optimiser des processus

Les algorithmes aident à trouver la meilleure solution possible à un problème. Par exemple :

- Dans les jeux vidéo, un algorithme peut calculer le chemin le plus court pour que le personnage atteigne sa destination.
- Dans les usines, un algorithme peut organiser la production pour minimiser les coûts.

02

Algorithmique : Notions de base

Structure Générale d'un algorithme

Algorithme Nom_algorithme	En-tête
Const : [Constantes] **Var :** [Variables]	Déclaration des constantes et des variables
Début [Instructions] **Fin**	Traitements

Notion d'objet

- **Nature d'objet:**
  - Un objet peut être de nature **constante ou variable** (il ne peut pas être les deux à la fois).
    - **Variable :** sa valeur peut changer au cours de l'algorithme.
    - **Constante :** sa valeur est fixée tout au long de l'algorithme.

- **Type d'objet:**
  - Un type définit l'ensemble des valeurs que peut prendre un objet de ce type ainsi que les actions qui y sont autorisées.
  - Un type peut être classé selon plusieurs critères parmi lesquels :
    - Simple/composé
    - Prédéfini/ personnalisé :
      - Prédéfini : s'il est défini avec le langage utilisé. Il existe quatre types standard : entier, réel, booléen, caractère.
      - Personnalisé : s'il n'est pas prédéfini. Il doit être alors défini dans le bloc TYPE.

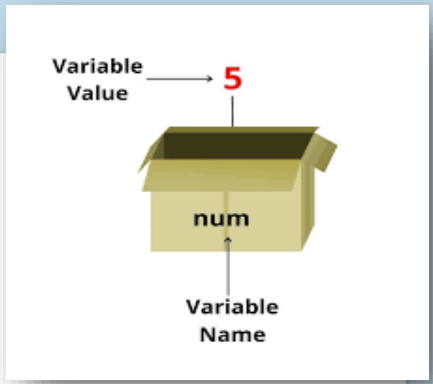
03

Les Variables

Qu’est-ce qu’une variable ?

Imaginez que vous avez des **boîtes** pour ranger différents objets.
Une **boîte pour les pommes** 🍎
Une **boîte pour les lettres** 🅰️
Une **boîte pour les phrases** 📄
Chaque boîte peut contenir **un seul type d’objet**.
👉 En algorithmique, c’est **pareil** : une **variable** est comme une **boîte** qui contient une **valeur** qu'on peut **lire**, **changer** ou **afficher**.

Notion de variable:

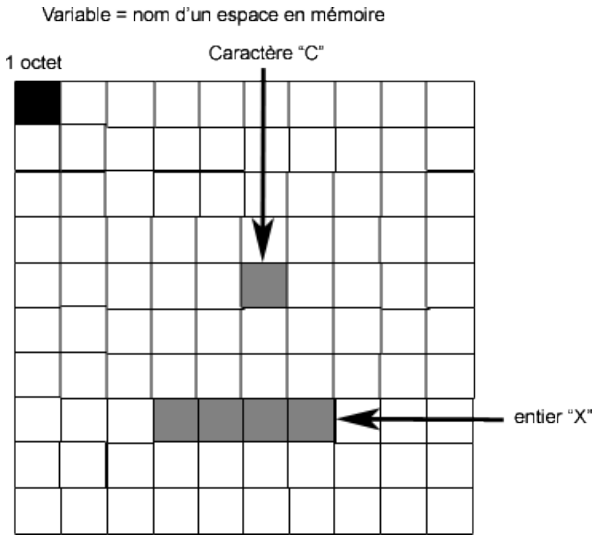


Dans les langages de programmation une **variable** sert à stocker la valeur d'une donnée

- Une variable désigne en fait un emplacement mémoire dont le contenu peut changer au cours d'un programme (d'où le nom variable)
- Règle : Les variables doivent être **déclarées** avant d'être utilisées, elle doivent être **<u>caractérisées par :</u>**
  - **un nom (Identificateur)**
  - **un type (entier, réel, caractère, chaîne de caractères, …)**
  - **Une valeur**

variable

- Un nom
- une valeur
- un type



(la valeur d'une variable peut changer au cours de l'exécution).

Déclaration : **<variable> : <type>**

Exemple Variable A, B :entier
d:réel

Une expression, pour un type, est soit une constante, soit une variable, soit constituée à l'aide de constantes, de variables, de parenthèses et des opérateurs.

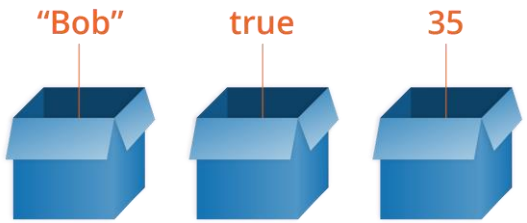
Syntaxe générale de Déclaration d’une variable :

🛠 **3. Déclaration d’une variable**

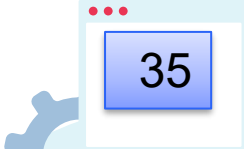
<u>Syntaxe générale :</u>

VARIABLE nom_variable : TYPE

Les types de variables



Type de variable	Ce qu'elle contient	Exemple
Entier	Des nombres entiers	5, -10, 0, 200
Réel	Des nombres avec virgule	3.14, -2.5, 0.99
Caractère	Un seul caractère (lettre, chiffre)	'A', 'z', '9'
Chaîne de caractères	Du texte	"Bonjour", "123", "oui"
Booléen	Vrai ou Faux	Vrai, Faux



🧱 2. Les types de variables

Type de variable	Nom technique	Contenu	Exemples	Plage de valeurs
Nombre entier	`ENTIER` ou `INTEGER`	Des nombres sans virgule	3, -7, 0, 150	En général : de -2 147 483 648 à 2 147 483 647
Nombre réel	`REEL` ou `REAL`	Nombres avec virgule	3.14, -0.5, 100.0	Dépend de la machine, mais permet beaucoup de décimales
Caractère	`CARACTERE` ou `CHAR`	Une seule lettre, chiffre ou symbole	'A', 'b', '9', '@'	Tous les caractères possibles (lettres, chiffres, symboles)
Chaîne de caractères	`CHAINE` ou `STRING`	Suite de caractères	"Bonjour", "123abc"	Dépend du langage, parfois limité à 255 caractères
Booléen	`BOOLEEN` ou `BOOLEAN`	Vrai ou Faux	VRAI, FAUX	Deux valeurs possibles

Type Entier (Integer)

Des **nombres entiers**, **positifs ou négatifs, sans virgule.**

🧮 **Exemples** : -10, 0, 25, 2023

🧪 **Exemple en algorithme :**

Variable age : entier
age ← 20

Le type Réel (Réel)

📦 Boîte qui contient des **nombres avec virgule**.

- 🧮 **Exemples** : 3.14, -2.5, 0.0

🧪 **Exemple en algorithme :**

Variable temperature : réel

temperature ← 36.5

Types numériques (Récapitulation)

Type Numérique	Plage
Byte (octet)	0 à 255
Entier simple	-32 768 à 32 767
Entier long	-2 147 483 648 à 2 147 483 647
Réel simple	$-3{,}40 \times 10^{38}$ à $-1{,}40 \times 10^{45}$ pour les valeurs négatives $1{,}40 \times 10^{-45}$ à $3{,}40 \times 10^{38}$ pour les valeurs positives
Réel double	$1{,}79 \times 10^{308}$ à $-4{,}94 \times 10^{-324}$ pour les valeurs négatives $4{,}94 \times 10^{-324}$ à $1{,}79 \times 10^{308}$ pour les valeurs positives

Type caractère

Un seul **caractère** (lettre, chiffre, symbole)

🧪 **Exemple en algorithme :**

Exemples : 'A', 'b', '7', '?'

Variable lettre : caractère

lettre ← 'A’

Lettre ← ‘f'

Symbole ← '?'

Les chaînes de caractères (chaîne, string)

Une **suite de caractères** (lettres, chiffres, mots…)

- **Exemples :** "Bonjour", "123", "Algorithme"

🧪 **Exemple en algorithme :**

Variable nom : chaîne

- nom ← "Paul "
- Message ← "Bonjour tout le monde !"

Booléen

Seulement **deux valeurs possibles** :

- VRAI (true)
- FAUX (false)

🧪 Exemple en algorithme :

- EstMajeur ← vrai
- AReussi ← faux

Table ASCII

- ASCII, c'est comme un **dictionnaire** qui associe **chaque caractère** (lettre, chiffre, symbole) à un **nombre**.

b_7 b_6 b_5 →					0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
Bits b_4 ↓	b_3 ↓	b_2 ↓	b_1 ↓	Column → / Row ↓	0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	—	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

EXERSICE :

exemple	Variable	Valeur	Type	Description
nom = "Alice"				
age = 25				
taille = 1.70				
est_majeur = True				

EXERSICE :

exemple	Variable	Valeur	Type	Description
nom = "Alice"	nom	Alice	Chaine de caractère	Chaîne de caractères représentant un nom.
age = 25	age	25	entier	Entier représentant l'âge
taille = 1.70	taille	1.70	réel	Nombre à virgule flottante représentant la taille
est_majeur = True	est_majeur	True .	Booléen	Valeur booléenne indiquant si la personne est majeure

Choix des identificateurs

Le choix des noms de variables est soumis à quelques **règles** qui varient selon le langage, mais en général :

- Un nom doit commencer par **une lettre alphabétique.**

Exemple **valide : A1**

exemple invalide : 1A

- Il doit être constitué uniquement de lettres, de chiffres et de soulignement (_). (Éviter les caractères de ponctuation et les espaces).

Valides : Info2011, Info_2011

Invalides : Info 2011, Info-2011, Info;2011

.

Remarque : **en pseudo-code algorithmique**, on va respecter les règles citées, même si on est libre dans la syntaxe des identificateurs

04

Les Constantes

Constantes

1. Une constante est une valeur fixe qui ne change jamais pendant l'exécution de l'algorithme.
2. Elle est souvent écrite en majuscules pour la distinguer.

$$\pi = 3.14$$

```
const

Nom_constante = valeur ;
```

```
const
        PI = 3.14;
```

```
CONST TAUX_TVA ← 0.19
```

05

Les Expressions et opérateurs

Expressions et opérateurs

Une expression est une combinaison de variables, constantes et opérateurs qui produit une valeur.

Les Opérateurs :

- **Arithmétiques :** +, -, *, /, MOD, DIV
- **comparaison:** == (égal), != (différent), <, >, <=, >
- **Logiques :** ET, OU, NON.

Comparaison	Opérateurs
=	Égal à
<	Inférieur à
>	Supérieur à
<=	Inférieur ou égal à
>=	Supérieur ou égal à
<>	Différent de

✨ a) Opérateurs arithmétiques

- Ceux qu'on utilise pour faire des calculs :

Opérateur	Signification	Exemple	Résultat
`+`	addition	`3 + 2`	`5`
`-`	soustraction	`5 - 2`	`3`
`*`	multiplication	`4 * 3`	`12`
`/`	division	`10 / 2`	`5`
`%`	modulo (reste)	`7 % 3`	`1` (reste)

🔍 b) Opérateurs de comparaison (relationnels)

- Ils comparent deux valeurs et donnent une réponse **vraie ou fausse** (true/false).

Opérateur	Signification	Exemple	Résultat
`==`	égal à	`5 == 5`	`vrai`
`!=`	différent de	`4 != 3`	`vrai`
`>`	supérieur à	`6 > 2`	`vrai`
`<`	inférieur à	`2 < 3`	`vrai`
`>=`	supérieur ou égal	`5 >= 5`	`vrai`
`<=`	inférieur ou égal	`3 <= 5`	`vrai`

⚙ c) Opérateurs logiques (booléens)

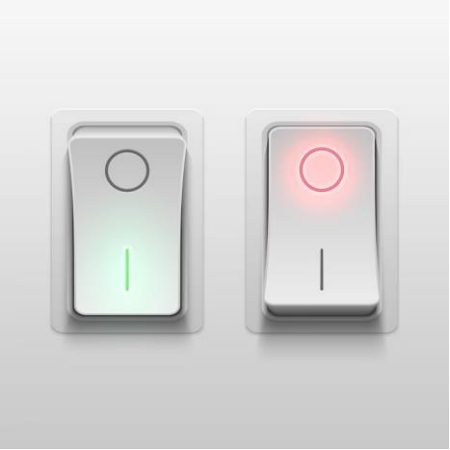
- Ils permettent de **combiner des conditions**.

Opérateur	Signification	Exemple	Résultat
`ET` ou `&&`	Vrai si les deux sont vraies	`5 > 3 ET 2 < 4`	`vrai`
`OU` ou `		`	Vrai si au moins une est vraie
`NON` ou `!`	Inverse le résultat	`NON (5 > 3)`	`faux`

🧠 **exemple:**

Imagine deux interrupteurs :

- **ET** = les deux interrupteurs doivent être ON pour allumer la lumière.
- **OU** = au moins un interrupteur ON suffit.
- **NON** = tu appuies sur un bouton pour faire l'inverse (ON devient OFF).



🎯 Priorité des opérateurs – Qui passe en premier ?

Les opérateurs suivent un ordre de priorité :

🏆 **Ordre de priorité (comme les règles de calcul en mathématiques) :**

1. **Parenthèses** ( )
2. **Multiplication** *, **Division** /, **Modulo** %
3. **Addition** +, **Soustraction** -
4. **Comparaisons** ==, !=, >, <, >=, <=
5. **Logiques** : NON → ET → OU

Comparaison	Opérateurs
=	Égal à
<	Inférieur à
>	Supérieur à
<=	Inférieur ou égal à
>=	Supérieur ou égal à
<>	Différent de

🧠 **Astuce :**
Utilise des **parenthèses** pour forcer l'ordre, comme en mathématiques.

1. 🔍 **Exemple :**
   3 + 5 * 2 = 3 + 10 = 13 (la multiplication est faite avant l'addition)
   (3 + 5) * 2 = 8 * 2 = 16

L’instruction d’affectation

L'opération consiste à affecter une valeur à une variable. Elle est représentée par une flèche orientée à gauche

Exemple :

1/ $A \leftarrow 15$

2/ $A \leftarrow B+3$

3/ $A \leftarrow A+5$

1/ Le terme de droite (15) est affecté au terme de gauche (variable A).
2/ Le terme de droite (valeur de la variable B + 3) est affecté au terme de gauche (variable A).
3/ Le terme de droite (valeur de A *avant l'instruction* + 1) est affecté au terme de gauche (variable A).
Dans ce dernier cas, la nouvelle valeur de A remplace l’ancienne.

Syntaxe:

variable ← expression

Exercice:

Quelles seront les valeurs des variables **A** et **B** après l'exécution des instructions suivantes ?

```
Variables A, B : Entier
Début
  A ← 5
  B ← 2
  A ← B
  B ← A
Fin
```

Résultat final :
A = 2
B = 2
⚠️ Remarque : Les valeurs **n'ont pas été échangées**, car à l'instruction A ← B, on a déjà perdu la valeur initiale de A (qui était 5).
Ensuite B ← A copie juste la nouvelle valeur de A (2).

Comment faire pour échanger les valeurs de A et B ?

Pour échanger les valeurs de deux variables sans les perdre, on utilise une variable temporaire

```
Variables A, B, Temp : Entier
Début
  A ← 5
  B ← 2
  Temp ← A
  A ← B
  B ← Temp
Fin
```

Exercice 2

Quelles seront les valeurs des variables A, B et C après l'exécution des instructions suivantes ?

Variables A, B, C Entier
Début
  A ← 5
  B ← 3
  C ← A + B
  A ← 2
  C ← B - A
Fin

Variable	Valeur finale
A	2
B	3
C	1

```
int x = 5, y = 10;
bool Test1 = (x < y);  // Test1 = True
bool Test2 = (x == y); // Test2 = False
```

06

Les instructions d'entrées / Sorties

Lecture et écriture

- **Écriture :** communiquer des valeurs à l'utilisateur en les affichant à l'écran.

Écrire : "Entrez votre nom : "

- **Lecture :**

entrer des valeurs au clavier pour qu'elles soient utilisées par le programme.

Lire : NomFamille

4. 🔁 Instructions d'entrée (lecture) et de sortie (écriture)

- **a) 📜 Écriture (afficher un résultat)**
  En pseudo-code :

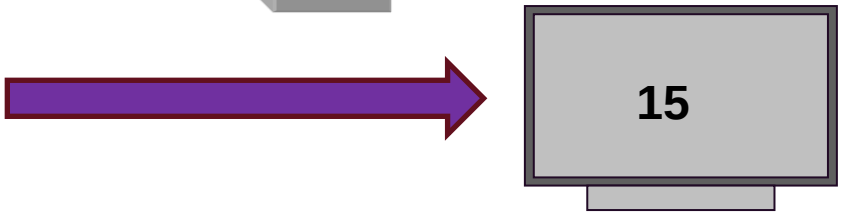
```
Écrire ("Bonjour")
Écrire (a)
Écrire ("Le résultat est : ", resultat)
```

Écrire = **parler à l'utilisateur**, afficher quelque chose à l'écran.

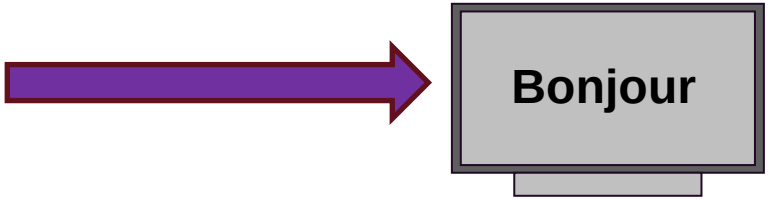
Exemples :

x

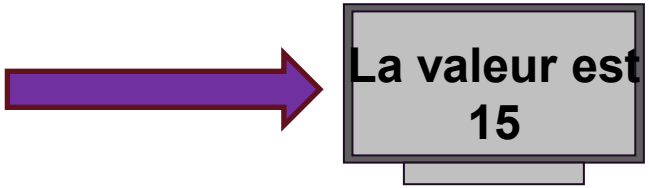
- Écrire (X)



- Écrire("Bonjour")



- Écrire("la valeur est" ,X)



Merci pour votre attention